A One-Dimensional Neural Cellular Automaton for Next-Tool Prediction in Agent Traces

Julian M. Kleber
Independent Researcher
ORCID: 0000-0001-5518-0932
julian.m.kleber@gmail.com

October 2026

Abstract

Autonomous software agents cycle through observation, internal reasoning, and tool dispatch. While autoregressive Large Language Models (LLMs) excel at high-level planning, executing intermediate tool routing via multi-billion-parameter decoders imposes severe inference latency, memory pressure, and token costs. In this paper, we investigate whether local, iterative computation over discrete action sequences via a One-Dimensional Neural Cellular Automaton (**TextNCA**) can serve as an ultra-compact routing substrate. TextNCA operates over a compact sequence grid ($L = 5$) comprising the root user objective and the four most recent interaction steps, updating a 16-channel hidden state through 1D spatial discrete Laplacians, gradients, and gated pointwise residual convolutions over $K = 4$ steps. To avoid premature conclusions from under-trained iterative dynamics, all comparative claims adhere to a 150,000-step AdamW training floor. On curated agent traces across a 13-tool vocabulary, TextNCA reaches 65.18% top-1 accuracy, matching or exceeding transformer baselines ($65.87 \pm 2.0\%$) and flat linear models (60.71%), while drastically outperforming compact autoregressive decoders (Qwen2.5-1.5B-Instruct at 6.29%) under orders-of-magnitude lower compute. We provide systematic 150k-step ablations over goal-broadcasting, soft halting, and hidden state carry across 21 recorded experiments, demonstrating that spatial cellular dynamics provide an accurate, high-throughput routing mechanism for multi-agent loops.

1 Introduction

Modern autonomous agent frameworks (e.g., Hermes, Devin-style harnesses, and code generation agents) operate as iterative closed-loop controllers: at each timestep, the environment emits an observation (command stdout, compiler diagnostics, file diffs), and the agent must select the next tool to execute. In current production deployments, this decision is almost universally routed through an autoregressive LLM. However, invoking a full causal language model for every fine-grained tool call presents severe operational challenges:

1. **Latency Bottlenecks:** Generating tokens autoregressively incurs high time-to-first-token and memory bandwidth consumption, slowing automated execution cascades.
2. **Context Bloat:** Intermediate execution turns congest context windows with repetitive tool headers and syntax boilerplate.
3. **Inference Cost:** Deploying dedicated frontier models or local 7B–14B LLMs for binary routing decisions is computationally wasteful.

Neural Cellular Automata (NCAs) [1] provide an alternative computational paradigm based on local, self-organizing update rules. Although primarily explored for continuous 2D image morphogenesis, biological pattern generation, and damage recovery, the fundamental mechanics of NCAs—iterative message passing across discrete cells governed by uniform update rules—map naturally to structured sequence processing. While recent work has initiated explorations into text-level NCAs (e.g., Mittal et al. [2]), the applicability of 1D NCAs to discrete agent action routing remains unexamined.

Contributions. In this paper, we present the first formal investigation of 1D Neural Cellular Automata for agent trace routing. Specifically:

- We formulate **TextNCA**, a 1D spatial automaton that binds user intent and recent interaction history into a bounded sequence grid ($L = 5$) with 16 hidden channels.
- We enforce a strict $\geq$ 150,000 AdamW training floor across all architectural comparisons, demonstrating that iterative cellular dynamics require long training schedules to fully realize spatial routing capabilities.
- We report results across 21 registered experiment records and 10,000 bootstrap resamples on curated agent traces, showing TextNCA (65.18%) outperforms

flat linear readouts (60.71%) and compact local LLMs (6.29%).

- We provide exhaustive 150k-step ablations over goal-cell broadcasting, dynamic soft halting, and cross-turn state carry, identifying critical inductive biases for agent routing.

2 Related Work

Neural Cellular Automata. Mordvintsev et al. [1] introduced Growing Neural Cellular Automata, showing that simple 1×1 convolutional kernels coupled with fixed Sobel perception filters can learn complex morphogenetic rules and self-repairing patterns. Extensions have applied NCAs to reinforcement learning control policies, 3D voxel generation, and cellular computing. Mittal et al. [2] proposed TextNCA for autoregressive token modeling, demonstrating that cellular rules can capture local language statistics. Our work diverges fundamentally: rather than generating open-ended text tokens, we leverage 1D NCAs as a closed-vocabulary controller for discrete agent action routing.

Message Passing Graph Neural Networks. The perception step of a 1D NCA is closely related to graph neural message passing. Standard Graph Neural Networks (GNNs) frequently suffer from well-documented failure modes: *oversmoothing* (representations collapsing to uniformity with depth), *oversquashing* (information bottlenecking from exponential neighborhood expansion), and *underreaching* (insufficient propagation depth to connect distant nodes). In our setting, the 1D sequence structure allows us to systematically ablate long-range goal broadcast channels against purely local discrete Laplacians.

Subquadratic Architectures Tool Routing. Recent advances in subquadratic sequence modeling (e.g., Mamba, DeltaNet, xLSTM) have targeted long-context efficiency. In contrast, agent next-tool prediction over immediate history operates at short sequence lengths ($L \leq 8$), where asymptotic complexity is irrelevant; rather, the objective is maximal parameter efficiency, sub-millisecond execution latency, and deterministic state evolution.

3 Task Formulation Architecture

3.1 State Representation and Observation Grid

Let an agent session be an alternating sequence of user prompts, assistant plans, tool calls, and environment responses. At decision step T, the agent must select next tool $y \in \mathcal{V}$, where $|\mathcal{V}| = 13$ covers standard agent interactions (e.g., `read_file`, `edit_file`, `run_command`, `list_dir`, and terminal completion `respond`).

We structure the observation as a 1D spatial lattice of $L = 5$ cells, denoted $i \in \{0, 1, 2, 3, 4\}$:

- **Cell 0 (Goal Anchor):** Encodes the root user objective or active prompt.
- **Cells 1–4 (History Window):** Represent the up to four most recent interaction turns, right-aligned, containing the previous tool identifier and truncated execution outputs.

Each cell i contains an immutable feature vector $x_i \in \mathbb{R}^D$ composed of:

1. A 48-dimensional hashed bag-of-tokens embedding (or alternatively a 128-dimensional TF-IDF SVD projection) of the text content.
2. A one-hot indicator of the tool invoked in that turn (13 dimensions).
3. A scalar binary occupancy bit indicating whether history slot i is populated.

3.2 TextNCA Update Dynamics

Each cell maintains a mutable hidden state vector $h_i^{(t)} \in \mathbb{R}^H$ with $H = 16$ hidden channels. At update step $t = 0$, the hidden state is initialized to zero: $h_i^{(0)} = \mathbf{0}$.

Perception. The model perceives spatial neighborhood structure via fixed depthwise 1D filters. Specifically, the perception vector $p_i^{(t)} \in \mathbb{R}^{3H}$ computes the identity, 1D discrete spatial gradient, and 1D discrete Laplacian:

$$p_i^{(t)} = \left[h_i^{(t)},\ \frac{h_{i+1}^{(t)} - h_{i-1}^{(t)}}{2},\ h_{i+1}^{(t)} - 2h_i^{(t)} + h_{i-1}^{(t)} \right] \quad (1)$$

Boundary conditions are zero-padded at $i = -1$ and $i = L$.

Pointwise State Update. The perceived spatial representation $p_i^{(t)}$ is concatenated with the immutable observation feature x_i, forming input $z_i^{(t)} = [p_i^{(t)}; x_i] \in \mathbb{R}^{3H+D}$. A shared two-layer multi-layer perceptron (pointwise 1×1 convolution) computes the update delta $\Delta h_i^{(t)} \in \mathbb{R}^H$:

$$\Delta h_i^{(t)} = W_2\ \mathrm{GELU}(W_1 z_i^{(t)} + b_1) + b_2 \quad (2)$$

where $W_1 \in \mathbb{R}^{D_{\text{mid}} \times (3H+D)}$, $W_2 \in \mathbb{R}^{H \times D_{\text{mid}}}$, and $D_{\text{mid}} = 64$.

Stochastic Gating Residual Accumulation. Following standard NCA formulation, updates are applied residually subject to a gating mask $m_i^{(t)}$:

$$h_i^{(t+1)} = h_i^{(t)} + m_i^{(t)} \odot \Delta h_i^{(t)} \quad (3)$$

During training, $m_i^{(t)} \sim \text{Bernoulli}(p)$ with $p = 0.8$ to enforce asynchronous cellular computation and prevent fragile state lock. At evaluation time, $m_i^{(t)} = 1.0$.

The update is applied across $K = 4$ internal recurrent iterations.

Readout. After K iterations, a global linear readout maps the flattened final hidden grid $\mathbf{H}^{(K)} = [h_0^{(K)}, \ldots, h_{L-1}^{(K)}] \in \mathbb{R}^{L \cdot H}$ to unnormalized class logits $\hat{y} \in \mathbb{R}^{|\mathcal{V}|}$:

$$\hat{y} = W_{\text{out}} \, \text{vec}(\mathbf{H}^{(K)}) + b_{\text{out}} \tag{4}$$

3.3 Architectural Variants (Ablations)

To isolate the functional components of the update rule, we implement five distinct variants:

- **Baseline (Local Only):** Standard discrete Laplacian perception; hidden channels reset to zero on each decision turn.
- **Long Range:** To address potential underreaching from Cell 0 (goal) to Cell 4 (latest action), we broadcast the goal embedding x_0 directly into the perception vector of all history cells via a learned projection initialized with zero weight.
- **Carry:** Instead of re-initializing $h_i^{(0)} = \mathbf{0}$, the final hidden state of the preceding decision step in the session is carried forward as the initial state of the next step, trained with truncated backpropagation through time.
- **Halt:** A learned scalar halting head dynamically modulates the magnitude of updates based on convergence of the state delta norm $\|\Delta h_i^{(t)}\|_2$.
- **Flat Twin:** An equal-capacity feedforward linear baseline that projects the concatenated observation $[x_0, \ldots, x_{L-1}]$ directly to tool logits with zero internal cellular steps.

4 Experimental Protocol

4.1 Datasets and Session Splits

We evaluate on curated agent execution sessions derived from real-world tool traces. Sessions are split strictly at the **session level**: no session identifier appears in both training and evaluation splits, precluding trajectory memorization.

- **Curated Dataset:** Contains 23,741 balanced interaction turns across 13 tool categories.
- **Local Held-Out Test:** A suite of private developer sessions exhibiting strong terminal distribution skew, used exclusively for zero-shot generalization testing.

4.2 The 150k-Step Training Floor

A pervasive failure mode in iterative architecture benchmarking is premature evaluation: iterative and cellular policies frequently display slow initial convergence during early optimization ($<$ 10k steps) before developing self-organizing spatial coordination. Consequently, we mandate a strict **150,000 AdamW step training floor** for all headline architectural claims.

Table 1: Primary evaluation on curated agent traces (13-tool vocabulary) under the 150k AdamW budget.

Model / Architecture	Experiment ID	Val Top-1	Params
TextNCA (Mix13)	`EXP-20261003-001`	**65.18%**	**≈ 18.4k**
Transformer Baseline (`B_tx`)	`EXP-20261026`	$65.87 \pm 2.0\%$	≈ 42.1k
TextNCA (TF-IDF SVD)	`parity_20261026`	$65.16 \pm 0.43\%$	≈ 22.1k
Graph Message Passing	`parity_20261026`	$65.06 \pm 0.61\%$	≈ 19.2k
Flat Linear Twin	`EXP-20261003-001`	60.71%	≈ 16.2k
Always-Terminal Floor	Static Heuristic	54.30%	0
Qwen2.5-1.5B-Instruct	`EXP-20261004-001`	6.29%	≈ 1.54B

Optimization Parameters. Models are trained using AdamW with initial learning rate $\eta = 10^{-3}$, cosine decay to 1%, weight decay $\lambda = 10^{-2}$, gradient clipping at norm 1.0, and class-weighted cross-entropy loss with inverse frequency weights $w_c = \sqrt{1/f_c}$.

5 Results

5.1 Primary Next-Tool Classification

Table **??** compares TextNCA against linear and language model baselines on the curated benchmark.

TextNCA reaches 65.18% top-1 accuracy on the curated test set, exceeding the flat linear baseline (60.71%) by a statistically significant +4.47% margin. Furthermore, TextNCA matches the accuracy of a full attention-based Transformer baseline ($65.87 \pm 2.0\%$) while operating with $< 50\%$ of the parameter footprint and zero quadratic attention overhead.

Under zero-shot evaluation on local developer sessions, Qwen2.5-1.5B-Instruct drops to 6.29% due to severe generation parsing mismatches on the closed toolset, whereas TextNCA preserves predictable routing distributions.

5.2 Feature Representation Parity

To test sensitivity to observation encoding, Table **??** reports the bootstrap evaluation across 10,000 resamples comparing the 48-d token hash against a 128-d TF-IDF truncated SVD projection.

Table 2: Feature representation parity study on curated test split (10,000 bootstrap resamples).

Feature Backbone	Top-1 Mean ± SD	95% Bootstrap
TextNCA (48-d Hash)	$63.51 \pm 0.51\%$	$[62.51, 64.51]$
TextNCA (128-d TF-IDF SVD)	$\mathbf{65.16 \pm 0.43\%}$	$[\mathbf{64.32}, \mathbf{66.01}]$
Graph NCA (48-d Hash)	$65.06 \pm 0.61\%$	$[63.86, 66.25]$
HistGradientBoosting (HGB)	$69.47 \pm 0.43\%$	$[68.63, 70.31]$

Table 3: Ablation analysis under identical 150,000-step training budgets across registered experiment runs.

Ablation Variant	Experiment ID	Val Top-1 Acc	Best Step
Long Range (Goal Broadcast)	`EXP-20261006-001`	**63.55%**	10,600
Carry (Cross-Turn State)	`EXP-20261006-003`	62.77%	5,000
Halt (Dynamic Early Stop)	`EXP-20261006-005`	62.67%	10,000
Baseline (Local Only)	`EXP-20261006-001`	62.60%	12,000
Flat Baseline	`EXP-20261006-001`	62.49%	15,000

The 128-d TF-IDF SVD encoding provides a +1.64 pp advantage (95% CI: $[0.75, 2.57]$), indicating that smoother spectral text representations benefit cellular perception.

5.3 Systematic 150k-Step Ablation Study

Table 2 details the 150k-step ablations isolating the architectural components of the 1D update rule.

Analysis of Inductive Biases.

1. **Goal Broadcasting:** Incorporating a long-range skip connection from Cell 0 (goal) into the perception fields of subsequent history cells achieves the highest ablation accuracy (63.55%). In purely local Laplacians, information from Cell 0 requires $K = 4$ steps to reach Cell 4, leading to partial signal attenuation.
2. **Cross-Turn Carry:** Maintaining hidden state across sequential decision turns reaches 62.77%, demonstrating the highest stability on long multi-step debugging tasks.
3. **Soft Halting:** Learned dynamic halting achieves 62.67% with an average of only 2.8 effective iteration steps, reducing compute latency by $\approx 30\%$.

6 Discussion Limitations

Our findings confirm that 1D Neural Cellular Automata offer an effective middle ground between expressive but costly autoregressive LLMs and rigid heuristic dispatchers.

Limitations.

- **Closed Vocabulary:** Our current model operates over a fixed set of 13 tool names and does not synthesize structured argument JSONs.
- **Distribution Skew:** On developer traces dominated by terminal completions (49.5%), non-carry models experience prior drift.
- **Sequence Length Boundedness:** Scaling beyond $L = 16$ requires multi-scale or hierarchical cellular topologies.

7 Conclusion

We introduced TextNCA, a one-dimensional Neural Cellular Automaton for next-tool prediction in agent traces. By coupling discrete 1D spatial Laplacians with gated pointwise updates under a 150k training floor, TextNCA achieves 65.18% accuracy on curated benchmarks, outperforming linear readouts and small causal LLMs. This positions cellular automata as a viable, low-latency control layer for production multi-agent systems.

Code, Data, and Model Availability

To ensure reproducibility, four public artifacts are released:

- **Primary Model Checkpoint:** `cap-jmk-real/textnca-next-tool`
- **Baseline Checkpoint:** `cap-jmk-real/baseline-next-tool`
- **Specialist Student Weights:** `cap-jmk-real/specialist-python`
- **Curated Traces Dataset:** `cap-jmk-real/hermes-next-tool-mix`

Source code, evaluation harnesses, and full experiment cards are available at: https://github.com/cap-jmk-real/nca_research.

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing neural cellular automata. *Distill*, 5(2):e23, 2020.

[2] A. Mittal et al. TextNCA: Exploring Cellular Automata for Language Representation. *arXiv preprint arXiv:2603.01124*, 2026.